

Java NIO (File IO) - Part2 and 3

In Part-1, we already discussed enhancements NIO(File I/O) provided compare to Stream based File Handling.

Today, lets cover the implementation part.

2 important parts of NIO(File I/O) are:

1. Buffer
2. Channel

In Buffer:-

- Data always flows in Buffer.
- We create and manage the buffer (NOT the framework).

In Channel :-

- Consider it as replacement of Stream (FileInputStream &FileOutputStream).
- Channel is the one which actually moves the data.

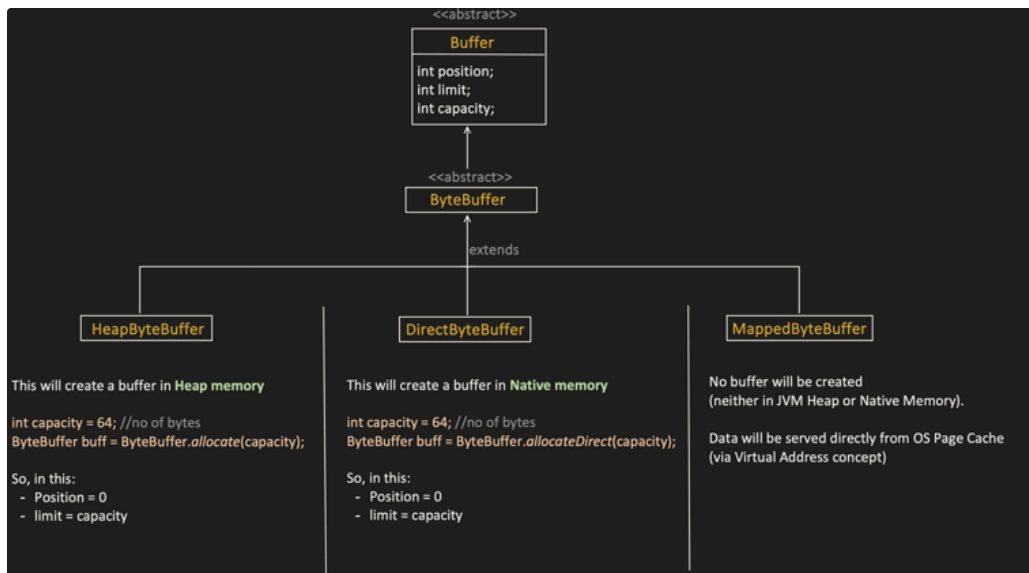
During Write:

Application fills Buffer → Channel drains the buffer(reads the data and send it down to) → OS → disk

During Read:

Application pass empty buffer to → Channel, goes to → OS/Disk, fetches the data → Channel fills Buffer and pass to → Application, drains buffer

1. Understand Buffer in NIO (File I/O)



Now lets see:

- how to WRITE and READ from buffer.
 - note: Write and Read method definition would be same for all different types of buffer, only internal implementation will be changed.
- And also lets see the use of fields present in buffer:
 - position
 - limit
 - capacity.

Capacity: total size of buffer. Never changes after creation.

Position: where the next read/write will happen. Moves forward automatically.

Limit: consider it as boundary (exclusive). We cannot read/write beyond limit.

Position < limit <= capacity

Example:

ByteBuffer buff = ByteBuffer.allocate(6); //creating HeapByteBuffer

[_,_,_,_,_,_]

Position = 0

Limit = 6 (boundary which is exclusive)

Capacity = 6 (total buffer size)

buff.put((byte)10); //put(..) method to write in buffer

buff.put((byte)20);

buff.put((byte)30);

[10 , 20 , 30 , _ , _ , _]

Position = 3

Limit = 6

Capacity = 6

Now lets say we have to read the data which is present withing the buffer.

buff.get(); //get() method to read from buffer

This will read position = 3 and it will return garbage.

So, we have to go back at beginning and that's where limit field use comes into picture.

buff.flip(); *Understand that this will switch our buffer from WRITE to READ mode.*

Limit = 3 (limit = position)

Position = 0

Capacity = 6

buff.get(); //returns 10

position = 1

buff.get(); //returns 20

position = 2

buff.get(); //returns 30

position = 3

buff.get()

position == limit, so no further READ is allowed.

Now say, we want to read the data again from start (second time I want to read), we can call "flip()" method again or call "rewind()".

That will reset pointers (flip changes both pointers limit = position, position = 0, whereas rewind touches only 1 i.e position=0)

After reading, to reuse the buffer again for write, we need to do:

buff.clear() ; *//Understand that this will switch our buffer back from READ to WRITE mode, but reset all pointers.*

Everything will get reset:

Position = 0

Limit = 6 (boundary which is exclusive)

Capacity = 6 (total buffer size)

Now, we understand what is buffer and how to write and read byte from the buffer (*by using put(..) and get() method*)

But wait:

One doubt might be coming at this point of time that, okay I understand how to write and read byte from buffer.

But what about if I have to write/read data as:

- int
- double
- long

etc.

I can not do, for example:

Say we have to store: int 65

int = 4 bytes

With ByteBuffer, we have to manually write each byte of integer '65'.

```
ByteBuffer bb = ByteBuffer.allocate(4);
```

```
bb.put((byte) 0);
```

```
bb.put((byte) 0);
```

```
bb.put((byte) 0);
```

```
bb.put((byte) 65);
```

And also while reading, we have to read all 4 bytes, club together to make int back.

That's an overhead.

That's similar to DataInputStream use case in Stream (Traditional File Handling).

In NIO (File I/O), we can do:

Approach-1

Approach-2

In Approach-1

```
1  ByteBuffer bb = ByteBuffer.allocate(100);
2
3  IntBuffer ib = bb.asIntBuffer(); //same bytes memory but different
   interpretation
4
5  ib.put(65);
6
7  ib.flip(); //switch from Write to read mode
8
9  int val = ib.get();
10
11
12 System.out.println(val); // 65
```

So ByteBuffer provides different typed views over the same bytes

(or in simple word, same bytes different interpretation):

```
1  ByteBuffer bb = ByteBuffer.allocate(100);
2
3  CharBuffer cb = bb.asCharBuffer(); // 2 bytes per element
4
5  ShortBuffer sb = bb.asShortBuffer(); // 2 bytes
6
7  IntBuffer ib = bb.asIntBuffer(); // 4 bytes
8
9  LongBuffer lb = bb.asLongBuffer(); // 8 bytes
10
11 FloatBuffer fb = bb.asFloatBuffer(); // 4 bytes
12
13 DoubleBuffer db = bb.asDoubleBuffer(); // 8 bytes
14
15
```

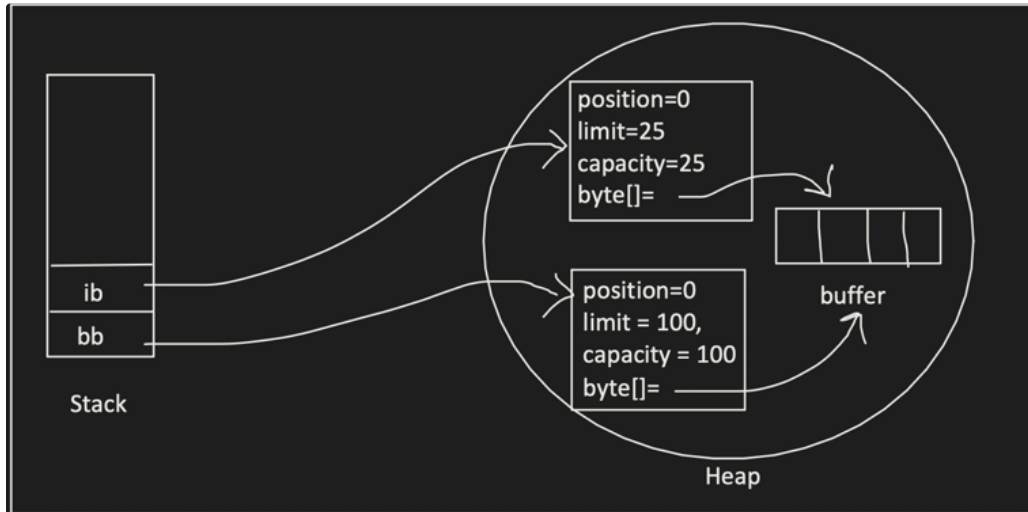
One very important point:

```
1  ByteBuffer bb = ByteBuffer.allocate(100);
2
3  IntBuffer ib = bb.asIntBuffer();
```

When we do this, ByteBuffer and IntBuffer both points to same memory of bytes.

But they maintain their own position, limit and capacity fields.

Integer, write and read bytes in group of 4, so capacity it will do : $100/4 = 25$



Channel:

- Works with only ByteBuffer (not IntBuffer, LongBuffer etc.)
- Also Buffer should be in READ mode, as it drains the buffer and send it to OS.

Write flow:

Application fills Buffer → Channel drains the buffer → OS/disk

So, when we do this:

```
1 ByteBuffer bb = ByteBuffer.allocate(100);  
2  
3 IntBuffer ib = bb.asIntBuffer();  
4  
5 ib.put(65);
```

Only ib (IntBuffer) position is changing, not bb (ByteBuffer).

So,

ib.position = 4

But

```
bb.position = 0
```

And before sending it to Channel, we need buffer to be in READ MODE and Channel only accept ByteBuffer.

So if we do:

```
bb.flip();
```

internally it will do:

```
bb.limit=bb.position
```

```
bb.limit=0
```

```
bb.position=0
```

So this is wrong.

So before we send it to Channel, we need to update the positions of ByteBuffer too.

```
1 ByteBuffer bb = ByteBuffer.allocate(100);  
2  
3 IntBuffer ib = bb.asIntBuffer();  
4  
5 ib.put(65);  
6  
7 bb.position(ib.position() * 4);  
8  
9 bb.flip();
```

In Approach 2:-

```
1 ByteBuffer bb = ByteBuffer.allocate(100);  
2  
3 bb.putInt(65);  
4  
5 int val = bb.getInt();  
6  
7 System.out.println(val); // 65
```

So no different Object is created to handle Int or Long or double etc.

ByteBuffer only does the encoding / decoding.


```
ByteBuffer bb = ByteBuffer.allocate(100);
```

Put method	Get method
bb.putInt(65);	bb.getInt();
bb.putShort((short)100);	bb.getShort();
bb.putLong(100L);	bb.getLong();
bb.putChar('A')	bb.getChar();
bb.putFloat(10.5f);	bb.getFloat();
bb.putDouble(10.5);	bb.getDouble();

So, there is no problem like in Approach-1, so update the position of ByteBuffer.

```
1 ByteBuffer bb = ByteBuffer.allocate(100);  
2  
3 bb.putInt(65);  
4  
5 bb.flip();
```

Another doubt might be coming at this point of time that: Hey, in previous topic : NIO advantages over Traditional Stream file handling. You mentioned that, we can READ and WRITE at random position.

How to do that?

Random Read can be done at 2 level:

Buffer Level

Channel Level

In Buffer Level

Use

- put(index, value)
- get(index)

But they bypass position and give us direct memory control.

```
1  ByteBuffer bb = ByteBuffer.allocate(16);
2
3  bb.putInt(0, 100);    // write at index 0
4
5  bb.putInt(4, 200);    // write at index 4
6
7  bb.putInt(8, 300);    // write at index 8
8
9  //NOW WE HAVE TO READ SPECIFIC POSITIONS WE WANT
10
11 int a = bb.getInt(0);  // 100
12
13 int b = bb.getInt(4);  // 200
14
15 int c = bb.getInt(8);  // 300
16
17
18 //no flip() required, as position, limit fields are not even
   maintained (no position movement).
```

One practical (real world) scenario I will tell, where these random methods are used.

when we do:

```
int b = bb.getInt(); // this will return the value + increment the position
```

But when we do:

```
int b = bb.getInt(1); //this works like a peek() method, returns the value but position is
unchanged. So can be used for peeking, validation etc.
```

Channel Level: Will cover when we will see "FileChannel"

With this I hope, Buffer is clear.

Now lets understand Channel and also complete code for file writing and reading_(different use cases):

In NIO File I/O, FileChannel is the class used to read and write files.

A logo for FileChannel, consisting of the text "FileChannel" in a yellow, sans-serif font, centered within a black rectangular box with a thin white border.

In traditional Stream based File handling, we have many classes:

- FileInputStream
- FileOutputStream
- BufferedInputStream
- BufferedOutputStream
- FileReader
- FileWriter
- BufferedReader
- BufferedWriter

But in NIO File I/O only 1 i.e. FileChannel.

Demo of FileChannel - Basics:

Basic Use cases:

1. Usecase1: use channel as only WRITE
2. Usecase2: use channel as only READ
3. Usecase3: Random read, both Channel Level and Buffer Level
4. Usecase4: Use channel as both READ and Write along with writing and reading primitive data types.

```
1 import java.io.IOException;
2 import java.nio.ByteBuffer;
3 import java.nio.channels.FileChannel;
```

Concept&&Coding-ByShrayansh

```

4 import java.nio.charset.StandardCharsets;
5 import java.nio.file.Path;
6 import java.nio.file.StandardOpenOption;
7
8 public class FileChannelDemo {
9
10     public static void main(String[] args) throws IOException {
11
12         // Usecase1: use channel as only WRITE
13         Path filePath = Path.of("nio_demo_usecase1.txt");
14         FileChannel writeChannel = FileChannel.open(filePath,
15             StandardOpenOption.WRITE,
16             StandardOpenOption.CREATE,
17             StandardOpenOption.TRUNCATE_EXISTING);
18
19         String text = "Hello All, how are you!";
20
21         ByteBuffer writeBuf = ByteBuffer.allocate(text.length());
22
23         writeBuf.put(text.getBytes());
24
25         writeBuf.flip(); //before handing over to channel, change it to
        READ mode so that channel can read and send it to OS.
26
27         writeChannel.write(writeBuf);
28
29
30
31
32         // Usecase2: use channel as only READ
33
34         FileChannel readChannel = FileChannel.open(filePath,
35             StandardOpenOption.READ);
36
37         ByteBuffer readBuf = ByteBuffer.allocate(100); //buffer
        capacity 100 bytes
38
39         //file could be more than 100 bytes, but in 1 round it can read
        upto 100bytes (buffer size) only. So need this loop.
40         //so we need to pass EMPTY Buffer to channel, so that it can
        fill this buffer (WRITE)
41
42         while (readChannel.read(readBuf) != -1) {
43             readBuf.flip(); // switch to read-mode, as
        Application now drain the buffer
44             while (readBuf.hasRemaining()) {
45                 System.out.print((char) readBuf.get());
46             }
47             readBuf.clear(); // switch back to write-mode
48         }
49
50
51         // Usecase3: Random read : both Channel Level and Buffer
        Level
52
53         FileChannel randomReadChannel = FileChannel.open(filePath,
54             StandardOpenOption.READ);
55
56         //1.Channel Level Random Read
57         //FileChannel maintains the position(cursor), with every READ
        this position is updated to indicate where the next read will start.
58         randomReadChannel.position(16); //updated the position, so next read
        will start from this byte index.
59
60         ByteBuffer randomReadBuf = ByteBuffer.allocate(11);
61         randomReadChannel.read(randomReadBuf);

```

```

60         randomReadBuf.flip(); // switch to read-mode
61
62         //2. Buffer Level Random Read, within a buffer doing random
read
63         System.out.println((char) randomReadBuf.get(0));
64         System.out.println((char) randomReadBuf.get(9));
65
66
67
68         // Usecase4: Use Channel as both READ and Write, writing
Integer, Character
69
70         FileChannel bothReadWriteChannel =
FileChannel.open(filePath, StandardOpenOption.READ,
71
StandardOpenOption.WRITE,
72
StandardOpenOption.CREATE,
73
StandardOpenOption.TRUNCATE_EXISTING);
74
75         // Write
76         ByteBuffer buffWrite = ByteBuffer.allocate(32);
77         buffWrite.putInt(100); // 4 bytes
78         buffWrite.putChar('A'); // 2 bytes
79         buffWrite.put("1".getBytes()); //1byte
80
81         buffWrite.flip(); // prepare for read
82         bothReadWriteChannel.write(buffWrite);
83
84
85         // Read first 5 bytes
86
87         //since we are using the same channel, we need to update its
cursor position to start
88         bothReadWriteChannel.position(0);
89         ByteBuffer buffRead = ByteBuffer.allocate(32);
90         bothReadWriteChannel.read(buffRead ); //channel will fill
the buffer with 32 bytes
91         buffRead.flip(); // prepare for read
92         System.out.println("Int: " + buffRead.getInt()); //100
93         System.out.println("Char: " + buffRead.getChar()); //A
94         System.out.println("Byte: " + (char)buffRead.get()); //1
95
96     }

```

Demo of FileChannel - Advanced:

Advanced Use cases:

1. Usecase1: TransferTo
2. Usecase2: Memory Mapping
3. Character encoding use cases

Usecase1: transferTo

```
1 //Without ZERO-COPY
```

```

2 Path sourcePath = Path.of("transfer_source.txt");
3 Path destPath = Path.of("transfer_dest.txt");
4
5 //we have to read from source
6 FileChannel srcChannel = FileChannel.open(sourcePath,
StandardOpenOption.READ);
7
8 //we have to write to destination
9 FileChannel dstChannel = FileChannel.open(destPath ,
StandardOpenOption.WRITE,
10 StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING);
11
12
13 ByteBuffer buf = ByteBuffer.allocate(8192);
14 /*
15 Reading from source in our JVM buffer heap memory
16 */
17 while(srcChannel.read(buf) != -1) {
18     buf.flip();
19     while(buf.hasRemaining())
20     /*
21      Now from our application writing to destination path
22      */
23     dstChannel.write(buf); //drain fully before next read
24     buf.clear();
25 }
26

```

```

1 //with ZERO-COPY feature
2 Path sourcePath = Path.of("transfer_source.txt");
3 Path destPath = Path.of("transfer_dest.txt");
4
5 //we have to read from source
6 FileChannel srcChannel = FileChannel.open(sourcePath,
StandardOpenOption.READ);
7
8 //we have to write to destination
9 FileChannel dstChannel = FileChannel.open(destPath ,
StandardOpenOption.WRITE,
10 StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING);
11
12
13 long size= srcChannel.size(); //provide us the file size bytes
14 /*
15 Directly moved the file content from OS -> destination, our JVM is
bypassed.
16 */
17 Long transferred = srcChannel.transferTo(0, size, dstChannel); //0
= file reading position, size = no of bytes need to be transferred
18

```

Usecase2: Memory Mapping

```

1 Path mapFile = Path.of("mmap_demo.txt"); //lets say already has the
data
2
3 FileChannel ch = FileChannel.open(mapFile,
4                                     StandardOpenOption.READ,
StandardOpenOption.WRITE);
5
6 //Very important:

```

```

7 // - OS assigned Virtual address range and creates an entry in VIRTUAL
  MEMORY AREA
8 // - No file data is added in RAM yet.
9
10 //so in this I have created the virtual address range for file from
   byte index 0 to my file size. We can decide to map any part of the
   file (100, 200). So only bytes at index 100 to 200 will be mapped. And
   our virtual address range would be 100 only.
11 MappedByteBuffer mmap = ch.map(MapMode.READ_WRITE, 0, ch.size());
12
13 //Now to read the data, we don't need channel, our application (JVM)
   knows address of the data, it will simply say read me the data at this
   address.
14 //mmap.get(i): this is where actual read happens, and during first
   call, PAGE FAULT happens, OS will read from disk and put into RAM and
   update the page table.
15 for (int i = 0; i < mmap.limit(); i++)
16     System.out.print((char) mmap.get(i));
17
18 // WRITE : modifies the OS page cache directly.
19 // OS will write back to disk lazily , as it just mark the page
   cache dirty(means this page on disk is outdated and need to be pushed
   to disk).
20 // force() makes it immediate.
21 mmap.put(0, (byte) 'a');
22 mmap.put(1, (byte) 'b');
23 mmap.put(2, (byte) 'c');
24 mmap.force(); // flush page cache to disk now

```

Usecase3: Char Encoding and Decoding

In NIO File I/O this need to be handled manually.

```

1 Path file = Path.of("chars_demo.txt");
2
3 FileChannel ch = FileChannel.open(file, StandardOpenOption.READ,
4                                     StandardOpenOption.WRITE,
5                                     StandardOpenOption.CREATE,
6                                     StandardOpenOption.TRUNCATE_EXISTING);
7
8 // Multi-byte UTF-8: dollar sign '$' = 3 bytes in UTF-8
9 /*
10  Encoding part
11 */
12 ByteBuffer bb = StandardCharsets.UTF_8.encode("Price: 100$");
13 ch.write(bb);
14
15
16 ch.position(0);
17 CharsetDecoder decoder = StandardCharsets.UTF_8.newDecoder();
18
19 ByteBuffer byteBuf = ByteBuffer.allocate(3);
20 CharBuffer charBuf = CharBuffer.allocate(6);
21
22 while (ch.read(byteBuf) != -1) {
23     byteBuf.flip();
24     /*
25      Manually decoding part
26      */
27     decoder.decode(byteBuf, charBuf, false);
28     charBuf.flip();
29     System.out.print(charBuf);
30     charBuf.clear();
31 }

```

```
16 // this is important, say decoder need 2 more bytes to make char,  
    but that will come in next read,  
17 //so it moves the unprocessed byte to front and set position ahead  
    of it. So that write should not override the unprocessed byte.  
18     byteBuf.compact();}  
19 }  
20 .  
21 .  
22 .  
23
```